第30课 递归思想（二）——汉诺塔、阶乘

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：阶乘递归（入门级）

题目： 输入一个非负整数 n（0 ≤ n ≤ 20），输出 n 的阶乘。

讲解思路：

1. 先和学生一起回忆阶乘的定义：`5! = 5×4×3×2×1`
2. 用递归的眼光重新看：`5! = 5 × 4!`，`4! = 4 × 3!` ……
3. 出口是什么？`0! = 1`（0 个人排队只有 1 种方式——就是没有人排）
4. 带学生手动展开 `factorial(4)` 的调用链
5. 强调返回值的类型用 `long long`（20! 非常大）

```
#include <iostream>
using namespace std;

// 递归求阶乘
long long factorial(int n) {
    // 递归出口：0! = 1
    if (n == 0) {
        return 1;
    }
    // 递归公式：n! = n * (n-1)!
    return (long long)n * factorial(n - 1);
}

int main() {
    int n;
    cin >> n;
    cout << factorial(n) << endl;
    return 0;
}
```

课堂互动： 让学生在纸上画出 `factorial(4)` 的调用过程（画"楼梯图"），从上往下展开，再从下往上收回来。

```
factorial(4) 开始，问 factorial(3)
  factorial(3) 开始，问 factorial(2)
    factorial(2) 开始，问 factorial(1)
      factorial(1) 开始，问 factorial(0)
        factorial(0) 到达出口，返回 1
      factorial(1) 收到 1，返回 1×1 = 1
    factorial(2) 收到 1，返回 2×1 = 2
  factorial(3) 收到 2，返回 3×2 = 6
factorial(4) 收到 6，返回 4×6 = 24
```

例题 2：斐波那契数列递归（入门级）

题目： 输入一个正整数 n（1 ≤ n ≤ 30），输出斐波那契数列的第 n 项。

讲解思路：

1. 回忆兔子的故事：`1, 1, 2, 3, 5, 8, 13, 21, …`
2. 递归公式：`fib(n) = fib(n-1) + fib(n-2)`
3. 出口：`fib(1) = 1, fib(2) = 1`
4. 在黑板上画"树形图"展示 `fib(5)` 的调用过程
5. 让学生观察：`fib(3)` 被算了好几次！引出"递归有时候效率不高"——为后面学动态规划埋伏笔

```
#include <iostream>
using namespace std;

// 递归求斐波那契第 n 项
int fib(int n) {
    // 递归出口
    if (n == 1 || n == 2) {
        return 1;
    }
    // 递归公式
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n;
    cin >> n;
    cout << fib(n) << endl;
    return 0;
}
```

课堂互动： 让学生试一下 `fib(40)`，感受递归变慢的过程。再试 `fib(50)`——可能要等很久甚至算不出来！让学生亲身感受"递归也有缺点"。

```
fib(5) 的调用树:

          fib(5)
         /      \
      fib(4)    fib(3)
      /    \     /    \
   fib(3) fib(2) fib(2) fib(1)
   /    \
fib(2) fib(1)

可以看到 fib(3) 被计算了 2 次，fib(2) 被计算了 3 次!
n 越大，重复越多，所以越来越慢。
```

例题 3：汉诺塔（核心例题）

题目： 输入一个正整数 n（1 ≤ n ≤ 10），表示汉诺塔的盘子数量。输出把所有盘子从 A 柱移动到 C 柱的每一步操作，最后输出总步数。

样例输入： `3`

样例输出：

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
Total: 7
```

讲解思路：

1. 用实物（三叠书）再演示一遍 3 个盘的过程
2. 引导学生发现"三步法"：
   - 步骤 1-3：把上面 2 个盘从 A 搬到 B（C 当中转）
   - 步骤 4：把最大的盘从 A 搬到 C
   - 步骤 5-7：把上面 2 个盘从 B 搬到 C（A 当中转）
3. 写代码：`hanoi(n, from, aux, to)` 四个参数
4. 带学生手动模拟 `hanoi(2, 'A', 'B', 'C')` 的调用过程

```
#include <iostream>
using namespace std;

int cnt = 0;  // 记录总步数

// 汉诺塔递归
// n: 盘子数量
// from: 起始柱（盘子现在在哪）
// aux: 中转柱（借助哪根柱子）
// to: 目标柱（盘子要去哪）
void hanoi(int n, char from, char aux, char to) {
    // 递归出口：只有 1 个盘子，直接搬
    if (n == 1) {
        cout << from << " -> " << to << endl;
        cnt++;
        return;
    }

    // 第一步：把上面 n-1 个盘从 from 搬到 aux（借助 to）
    hanoi(n - 1, from, to, aux);

    // 第二步：把最大的盘从 from 搬到 to
    cout << from << " -> " << to << endl;
    cnt++;

    // 第三步：把 n-1 个盘从 aux 搬到 to（借助 from）
    hanoi(n - 1, aux, from, to);
}

int main() {
    int n;
    cin >> n;
    hanoi(n, 'A', 'B', 'C');
    cout << "Total: " << cnt << endl;
    return 0;
}
```

重点板书：参数角色互换表

调用	from（起点）	aux（中转）	to（终点）
原问题 `hanoi(n, A, B, C)`	A	B	C
第一步 `hanoi(n-1, A, C, B)`	A	C	**B**
第三步 `hanoi(n-1, B, A, C)`	**B**	A	C

> 口诀："第一步把终点和中转换一换，第三步把起点和中转换一换"

课堂互动： 让学生手动写出 `hanoi(2, 'A', 'B', 'C')` 展开过程：

```
hanoi(2, A, B, C):
  hanoi(1, A, C, B):    输出 A -> B
  输出 A -> C
  hanoi(1, B, A, C):    输出 B -> C

所以 2 个盘的输出是:
A -> B
A -> C
B -> C
共 3 步
```

例题 4：阶乘循环 vs 递归对比（拓展 ）

题目： 用循环的方式再写一遍阶乘函数，对比两种写法。

讲解思路： 让学生看到同一个问题的两种解法，理解各自的特点。

```
#include <iostream>
using namespace std;

// 方法一：递归
long long factorial_rec(int n) {
    if (n == 0) return 1;
    return (long long)n * factorial_rec(n - 1);
}

// 方法二：循环
long long factorial_loop(int n) {
    long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int n;
    cin >> n;
    cout << "递归结果：" << factorial_rec(n) << endl;
    cout << "循环结果：" << factorial_loop(n) << endl;
    return 0;
}
```

> 两个方法结果一样！但是递归的代码更"优雅"——只需要两行核心代码。而循环的代码更"直白"——一步一步乘上去。
>
> **简单问题两种都行；复杂问题（如汉诺塔）递归是王者！**

附：例题源代码与运行结果

例题 1：Factorial (Recursive)

```
// Input a non-negative integer n (0 <= n <= 20),
// output n! using recursion.
// Formula: n! = n * (n-1)!, base case: 0! = 1
#include <iostream>
using namespace std;

long long factorial(int n) {
    // base case: 0! = 1
    if (n == 0) {
        return 1;
    }
    // recursive formula: n! = n * (n-1)!
    return (long long)n * factorial(n - 1);
}

int main() {
    int n;
    cin >> n;
    cout << factorial(n) << endl;
    return 0;
}
```

Sample Input

```
5
```

Expected Output

```
120
```

例题 2：Fibonacci Sequence (Recursive)

```
// Input a positive integer n (1 <= n <= 30),
// output the n-th Fibonacci number.
// Formula: fib(n) = fib(n-1) + fib(n-2)
// Base cases: fib(1) = 1, fib(2) = 1
#if 0
#include <iostream>
using namespace std;

int fib(int n) {
    // base cases
    if (n == 1 || n == 2) {
        return 1;
    }
    // recursive formula
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n;
    cin >> n;
    cout << fib(n) << endl;
    return 0;
}
#endif
```

Sample Input

```
10
```

Expected Output

```
55
```

例题 3：Tower of Hanoi

```
// Input n (1 <= n <= 10), the number of disks.
// Output each move to transfer all disks from peg A to peg C,
// then output the total number of moves.
// Strategy: move top n-1 to aux, move largest to target,
//           move n-1 from aux to target.
#if 0
#include <iostream>
using namespace std;

int cnt = 0;  // total move counter

// n: number of disks
// from: source peg
// aux: auxiliary peg
// to: destination peg
void hanoi(int n, char from, char aux, char to) {
    // base case: only 1 disk, move it directly
    if (n == 1) {
        cout << from << " -> " << to << endl;
        cnt++;
        return;
    }

    // step 1: move top n-1 disks from source to auxiliary (using dest as helper)
    hanoi(n - 1, from, to, aux);

    // step 2: move the largest disk from source to destination
    cout << from << " -> " << to << endl;
    cnt++;

    // step 3: move n-1 disks from auxiliary to destination (using source as helper)
    hanoi(n - 1, aux, from, to);
}

int main() {
    int n;
    cin >> n;
    hanoi(n, 'A', 'B', 'C');
    cout << "Total: " << cnt << endl;
    return 0;
}
#endif
```

Sample Input

```
3
```

Expected Output

```
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C
Total: 7
```

例题 4：Factorial - Recursion vs Loop

```
// Compare recursive and iterative implementations of factorial.
#if 0
#include <iostream>
using namespace std;

// Method 1: Recursion
long long factorial_rec(int n) {
    if (n == 0) return 1;
    return (long long)n * factorial_rec(n - 1);
}

// Method 2: Loop
long long factorial_loop(int n) {
    long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int n;
    cin >> n;
    cout << "Recursive: " << factorial_rec(n) << endl;
    cout << "Iterative: " << factorial_loop(n) << endl;
    return 0;
}
#endif
```

Sample Input

```
10
```

Expected Output

```
Recursive: 3628800
Iterative: 3628800
```